

6. $O(2^n)$, Exponential Time

7. $O(n!)$, Factorial Time (the highest order)

Notice that big O uses the following notation: a capital O, followed by a pair of parentheses containing a description of the order. The capital O represents ORDER or ON THE ORDER OF. The n represents the size of the input data the code will work on. We pronounce $O(n)$ as “big oh of n ” or “big oh n .”

You don’t need to understand the precise mathematic meanings of words like LOGARITHMIC or POLYNOMIAL to use big O notation. I’ll describe each of these orders in detail in the next section, but here’s an oversimplified explanation of them:

- $O(1)$ and $O(\log n)$ algorithms are fast.
- $O(n)$ and $O(n \log n)$ algorithms aren’t bad.
- $O(n^2)$, $O(2^n)$, and $O(n!)$ algorithms are slow.

Certainly, you could find counterexamples, but these descriptions are good rules in general. There are more big O orders than the ones listed here, but these are the most common. Let’s look at the kinds of tasks that each of these orders describes.

A Bookshelf Metaphor for Big O Orders

In the following big O order examples, I’ll continue using the bookshelf metaphor. The n refers to the number of books on the bookshelf, and the big O ordering describes how the various tasks take longer as the number of books increases.

$O(1)$, Constant Time

Finding out “Is the bookshelf empty?” is a constant time operation. It doesn’t matter how many books are on the shelf; one glance tells us whether or not the bookshelf is empty. The number of books can vary, but the runtime remains constant, because as soon as we see one book on the shelf, we can stop looking. The n value is irrelevant to the speed of the task,